

VÕ TIỀN

Thảo luận kiến thức CNTT trường BK về KHMT(CScience), KTMT(CEngineering)
<https://www.facebook.com/groups/khmt.ktmt.cse.bku>



Cấu Trúc Rời Rạc (Toán Rời Rạc)

CTRR - HK242

BELLMAN-FORD ALGORITHM

Thảo luận kiến thức CNTT trường BK
về KHMT(CScience), KTMT(CEngineering)
<https://www.facebook.com/groups/khmt.ktmt.cse.bku>

Mục lục

1	Mô tả	2
2	Cách hiện thực và Chạy code	3



1 Mô tả

Mô tả các param

- **G** là **Ma trận trọng số (weighted matrix)** : Mỗi đơn vị trong ma trận biểu diễn khoảng cách tương ứng. Ví dụ $G[i][j]$ là khoảng cách từ thành phố i tới thành phố j . Trong trường hợp giá trị 0 nghĩa là không có đường đi trực tiếp từ thành phố i tới thành phố j .
- **numberOfVertices** là số lượng đỉnh trong đồ thị
- **startVertex** đỉnh bắt đầu
- **goalVertex** đỉnh kết thúc
- **Label** Một array chứa các giá trị khoảng cách ngắn nhất từ đỉnh bắt đầu đến mỗi đỉnh khác trong đồ thị. Giá trị này được cập nhật trong quá trình thực hiện thuật toán Bellman-Ford.
- **Prev** Một array chứa các đỉnh trước đó trên đường đi ngắn nhất từ đỉnh bắt đầu đến mỗi đỉnh khác trong đồ thị. Nó giúp xác định đường đi ngắn nhất từ đỉnh bắt đầu đến một đỉnh cụ thể.

```
void BF(int G[20][20], int num_vertices, char start_ver, int value_arr[20], int prev_arr[20]);
```

Mô tả hàm BF

1. Khởi tạo **Label[start] = 0**, tức khoảng cách từ đỉnh bắt đầu đến chính nó bằng 0.
2. Kiểm tra xem trong mảng **Label** có tồn tại phần tử nào đang mang giá trị -1 không.
 - Nếu có, gán giá trị **Label** của từng đỉnh bằng khoảng cách trực tiếp từ đỉnh bắt đầu **start** đến đỉnh đó, lấy dữ liệu từ ma trận trọng số **G**.
 - Đồng thời, gán **Prev** của các đỉnh này là đỉnh bắt đầu.
 - Với các đỉnh mà khoảng cách đến bằng -1 (không liên kết), **Prev** của đỉnh đó vẫn giữ nguyên là -1.
 - Sau khi thực hiện gán xong, kết thúc hàm.
3. Nếu không còn giá trị -1 trong **Label** (tức đã khởi tạo xong khoảng cách), thực hiện bước tiếp theo:
 - Sao chép mảng **Label** vào một mảng tạm thời để giữ nguyên giá trị cũ trong quá trình cập nhật.
4. Duyệt từng đỉnh đích **U** và với từng đỉnh trung gian **V**:
 - So sánh giá trị **Label'[V] + G[V][U]** với **Label'[U]**, trong đó **Label'** là mảng tạm giữ giá trị trước đó.
 - Nếu **Label'[V] + G[V][U] < Label'[U]** và **Label'[V]** khác -1 (tức có đường đi hợp lệ), cập nhật:
$$\text{Label}[U] = \text{Label}'[V] + G[V][U] \quad \text{và} \quad \text{Prev}[U] = V$$
 - Trong trường hợp có nhiều đỉnh trung gian **V** cho giá trị nhỏ hơn **Label'[U]**, ưu tiên chọn đỉnh **V** có ký tự Alphabet nhỏ hơn.

```
string BF_Path(int G[20][20], int num_vertices, char start, char end);
```

Mô tả hàm BF_Path

1. Khởi tạo các mảng **Label** và **Prev** với giá trị ban đầu là -1.
2. Gọi lặp lại hàm **BF** đủ số lần cần thiết (thường là bằng số đỉnh) để tính toán mảng **Prev** tối ưu nhất.
3. Kiểm tra xem có tồn tại đường đi từ **start** đến **end** hay không:
 - Nếu không tồn tại (ví dụ **Label[end]** vẫn là -1), trả về chuỗi **"No path exists"** và kết thúc.
4. Nếu tồn tại đường đi, dựa vào mảng **Prev** để lần ngược từ đỉnh kết thúc về đỉnh bắt đầu, ghép các đỉnh lại thành chuỗi đường đi theo thứ tự từ **start** đến **end**.



5. Chuỗi đường đi trả về không có dấu cách thừa ở cuối, các đỉnh được phân tách bởi dấu cách.

2 Cách hiện thực và Chạy code

Phân hiện thực bellman.cpp có thể thêm thư viện bất kì trong bellman.h không giới hạn

```
1 void BF(int G[20][20], int num_vertices, char start_ver, int value_arr[20], int  
   ↪ prev_arr[20]);  
2 string BF_Path(int G[20][20], int num_vertices, char start, char end);
```

1. Tải src về hiện tại đang cập nhật thêm test case sau hiện tại chỉ 3 test cơ bản
2. Chạy run.bat (đối với mac thì ./run.sh)
3. Kết quả

Building project bellman VOTIEN with command:

```
g++ -std=c++17 -o main.exe -Iinclude -Iunit_test -g main.cpp src\bellman.cpp unit_test\unit_t  
unit_test\testcase\bellman01.cpp unit_test\testcase\bellman02.cpp
```

<biên dịch đang chạy và nếu thành công thì sẽ tiếp tục>

Build successful!

To run tests, use one of the following commands:

1. Run all tests: main.exe
2. Run a specific test: main.exe nameFunctionUnitTest

4. Giải thích cách chạy test case:

- main.exe chạy tất cả các test case đã được biên dịch vào chương trình.
- main.exe nameFunctionUnitTest chỉ chạy riêng test case có tên nameFunctionUnitTest (tên này là tên hàm test trong code).